

Homework 4

CS/ECE 374-B Intro to Algorithms & Models of Computation

University of Illinois Urbana-Champaign

Spring 2025

Instructors: Abhishek Umrawal & Ivan Abraham

Maximum Points: 100

Due Date: Monday, March 10, 2025, 11:59 PM

Instructions

1. Answer **ALL** questions.
2. Show all the work.
3. Legibly scribe your solutions and upload them as individual problems on Gradescope.

Suggestions

While writing your answers and/or proofs, think about the following.

1. What is it that you are given? What is it that you are assuming? What is it that you are trying to prove?
2. How exactly does a step follow from the previous step? Assume nothing is apparent, even algebraic simplifications. Your reasoning should be clear and convincing.
3. If you are using a result from the class then state it clearly before using it. You should identify where exactly you need that result, cast your problem into the setup of the result, and then use it.
4. If you are using a result from outside the class then you should prove it before using it.

While writing and analyzing an algorithm, note the following.

1. An algorithm is a finite set of primitive and unambiguous instructions to solve a computing problem.
2. Unless otherwise stated, you should give the fastest possible algorithm, i.e., an algorithm with the tightest possible asymptotic time complexity bound.

Problems

1. A *polynomial* $f(x; [a_k, \dots, a_0])$ of degree k for the argument $x \in \mathbb{R}$ is defined as the following sum:

$$f(x; [a_k, \dots, a_0]) = a_k x^k + a_{k-1} x^{k-1} + \dots + a_0,$$

where $a_i \in \mathbb{R}, i = 1, \dots, k$, and $a_k \neq 0$ are called the *coefficients* of $f(x; [a_k, \dots, a_0])$.

The phrase ‘*given a polynomial*’ means having access to the coefficients and the argument of the polynomial.

Note that, given two *polynomials* $f(x; [\alpha, \beta])$ and $g(x; [\gamma, \delta])$ each of degree one for the same argument $x \in \mathbb{R}$:

$$f(x) = \alpha x + \beta \text{ and } g(x) = \gamma x + \delta, \alpha, \gamma \neq 0,$$

their product $f(x; [\alpha, \beta]) \cdot g(x; [\gamma, \delta])$ can be calculated as follows:

$$f(x; [\alpha, \beta]) \cdot g(x; [\gamma, \delta]) = m_1 x^2 + (m_3 - m_1 - m_2) x_1 + m_2, \quad (\text{Karatsuba's algorithm})$$

where $m_1 = \alpha \cdot \gamma, m_2 = \beta \cdot \delta, m_3 = (\alpha + \beta) \cdot (\gamma + \delta)$.

Note that ignoring multiplications involving the argument x , $f(x; [\alpha, \beta]) \cdot g(x; [\gamma, \delta])$ is computed using *three* multiplications, i.e., computing m_1, m_2 , and m_3 .

Given two polynomials $f(x; [a_{n-1}, \dots, a_0])$ and $g(x; [b_{n-1}, \dots, b_0])$ each of degree $n - 1$, give an efficient algorithm (extend the above strategy) to calculate their product and analyze its asymptotic time complexity.

Solution: Denote our input in the form of

$$f(x) = \sum_{i=0}^{n-1} \alpha_i x^i, g(x) = \sum_{i=0}^{n-1} \beta_i x^i.$$

We can use the formula for multiplying linear polynomials, described above, that used only three multiplications, and apply it to $f(x)$ and $g(x)$. This is done by treating them as “fake” polynomials of the variable $x^{n/2}$. We get the following:

$$M_1(x) = f_1(x) \cdot g_1(x), M_2(x) = f_2(x) \cdot g_2(x), \text{ and } M_3(x) = (f_1(x) + f_2(x)) \cdot (g_1(x) + g_2(x)).$$

Thus, we have

$$h(x) = f(x)g(x) = M_1(x)x^{n/2} + (M_3(x) - M_1(x) - M_2(x))x^{n/2} + M_2(x).$$

We conclude that computing $h(x)$ can be done in $O(n)$ time, and three recursive calls to multiplying three polynomials of degree $n/2$. We thus get the following recurrence on the running time of this algorithm:

$$T(n) = O(n) + 3T(n/2) = O(n^{\log_2 3})$$

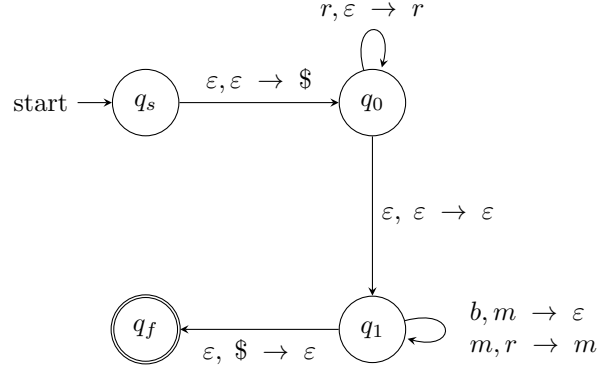
One can of course do better using FFT, but it is out of the scope of this class.

2. Willy's bakery shop is open for customers to place their cake orders. Willy uses a computer to calculate the total number of orders for each day and waits until all orders have been received. Willy then processes each cake by making it and then placing it in the oven. Let the set of actions be defined as $A := \{\mathbf{r}, \mathbf{m}, \mathbf{b}\}$, where:

- $\mathbf{r}$ represents registering an order,
- $\mathbf{m}$ represents making a cake, and
- $\mathbf{b}$ represents baking a cake.

Design a *pushdown automaton* $P = (Q, \Sigma, \Gamma, \delta, s, F)$ to help Willy keep track of the daily workload where $\Sigma \subseteq A$ and $\Gamma \subseteq A \cup \{\$\}$. Hint: Examples of acceptable sequences are $\mathbf{rmb}$ and $\mathbf{rrrmbmbmb}$.

Solution



3. Given two sorted integer arrays $A[1, \dots, m]$ and $B[1, \dots, n]$, describe and analyze an (as fast as possible) algorithm to compute the median of the $m + n$ elements in A and B .

Solution

We give the following recursive selection algorithm that we can use to find the median. We define $\text{Selection}(A, B, k)$ to be the k th largest value in the sorted arrays A and B . We can write $\text{Selection}(A, B, k)$ as follows:

0. If $m \leq 100$ or $n \leq 100$ then use brute force to merge the two arrays into C and return $C[k]$.¹
1. Let $i = \lfloor m/2 \rfloor, j = \lfloor n/2 \rfloor$. Take $a = A[i], b = B[j]$ as the middle element of arrays A, B respectively.
- 2a. If $k \leq (m + n)/2$:
 - i. If $a \leq b$, return $\text{Selection}(A[1, \dots, m], B[1, \dots, j], k)$
 - ii. Else, return $\text{Selection}(A[1, \dots, i], B[1, \dots, n], k)$
- 2b. Else:
 - i. If $a \leq b$, return $\text{Selection}(A[i + 1, \dots, m], B[1, \dots, n], k - i)$
 - ii. Else, return $\text{Selection}(A[1, \dots, m], B[j + 1, n], k - j)$

Then we can compute the median as the brute force median of $\text{Selection}(A, B, \lfloor (m+n)/2 \rfloor)$ and $\text{Selection}(A, B, \lceil (m+n)/2 \rceil)$.

Correctness: In every call of the algorithm, we eliminate some portion of the two arrays. Then it suffices to show that the k th element is never eliminated, so that eventually we are left with some small array containing our target and it is easy to find our median by brute force. This invariant is relatively simple to show.

Taking middle elements a, b of A, B , if $a \leq b$ then the left half of A is certainly in the left half of the combined arrays A and B . This comes from seeing that any element in the left half of A is less than every element in the right half of A and every element in the right half of B . Similarly, if $a > b$ then the right half of A is certainly in the right half of the combined arrays A and B . An identical argument applies to B .

We conclude that 2a and 2b in our algorithm never eliminate the k th element, so our algorithm terminates correctly.

Runtime: At each recursive call, we perform constant time comparison operations and then cut one of the two arrays in half. Our algorithm terminates when one array is whittled down to some small constant. At worst case, we have to fully exhaust both arrays, so we get asymptotic runtime $O(\log n + \log m)$ or $O(\log mn)$. Calling Selection twice doesn't change our asymptotic runtime.

Note: There exists an algorithm that runs in $O(\log(\min\{m, n\}))$ time, but full points are awarded for algorithms that match the provided solution in runtime.

¹Recall that merge is linear runtime with respect to the length of the smaller array, and selection of a sorted array is constant runtime, so we can say that this brute force takes constant time.

4. Given the alphabet $\Sigma = \{0, 1\}$, determine if each of the following languages is regular, context-free, or both.

- (a) $L_1 = \{0^i 1^j 0^k \mid j = i + k, i, j, k \geq 0\}$.
- (b) $L_2 = \{0^i 1^j 1^k \mid j = i + k, i, j, k \geq 0\}$.
- (c) $L_3 = \{\text{any string that does not contain } 0^n 1^n \text{ as a substring, } n \geq 1\}$.
- (d) $L_4 = L((0 + 1)^* 0^n 1^n, n \geq 0)$.

Solution

- (a) L_1 :
Not regular, prove by fooling set $F = 0^*$
For $x = 0^i, y = 0^j, i \neq j$
 $z = 1^{2i} 0^i$, then $xz \in L_1$ but $yz \notin L_1$
So F is an infinite fooling set for L_1 , L_1 is not regular.
Context-free, prove by the following CFG:
 $S \rightarrow AB \quad A \rightarrow 0A1 \mid \epsilon \quad B \rightarrow 1B0 \mid \epsilon$
- (b) L_2 :
Not regular, prove by fooling set $F = 0^*$
For $x = 0^i, y = 0^j, i \neq j$
 $z = 1^{3i} = 1^{2i} 1^i$, then $xz \in L_1$ but $yz \notin L_1$
So F is an infinite fooling set for L_1 , L_1 is not regular.
Context-free, prove by the following CFG:
 $S \rightarrow AB \quad A \rightarrow 0A1 \mid \epsilon \quad B \rightarrow 11B \mid \epsilon$
- (c) L_3 :
Regular, prove by the following regular expression.
 $L_3 = 1^* 0^*$
Context-free, because every regular language is also context-free.
- (d) L_4 :
Regular, prove by the following regular expression.
 $L_4 = (0 + 1)^*$
Context-free, because every regular language is also context-free.

5. Assume that the sum and product of two real numbers *each* can be computed in constant ($O(1)$) time.

Given two matrices $\mathbf{A}$ and $\mathbf{B}$ of real numbers each of size $n \times n$, one can compute the product of $\mathbf{A}$ and $\mathbf{B}$ in $O(n^3)$ time using a *brute-force* algorithm (three nested **for** loops each of size n). However, using a strategy similar to the Karatsuba's algorithm, mathematician Volker Strassen proposed a *divide-and-conquer* algorithm to calculate the product of $\mathbf{A}$ and $\mathbf{B}$ in $O(n^{2.8074})$ time.

Given a *black box* $\text{Straessen}(\mathbf{A}, \mathbf{B})$ that computes the product of $\mathbf{A}$ and $\mathbf{B}$ in $O(n^{2.8074})$ time, do the following.

- (a) Given a matrix $\mathbf{A}$ of real numbers of size $n \times n$ and an integer $k \in \mathbb{N}$, give an *efficient* algorithm to compute its ' k th power of 2' defined as $\mathbf{A}^{2^k}$.
- (b) Obtain the asymptotic time complexity bound for your algorithm from Part a. in terms of $m = 2^k$.
- (c) How would you modify your algorithm from Part a. to do the following. Given a matrix $\mathbf{A}$ of real numbers of size $n \times n$ and an integer $m \in \mathbb{N}$, give an *efficient* algorithm to compute $\mathbf{A}^m$, i.e. m , need not be 2^k for some $k \in \mathbb{N}$.
- (d) Obtain the asymptotic time complexity bound for your algorithm from Part c. in terms of m .

Note: You *cannot* use any additional knowledge of Linear Algebra to solve this problem.

Solution

- (a) The key idea is that $\mathbf{A}^{2^{k+1}} = \mathbf{A}^{2^k} \mathbf{A}^{2^k}$, from which the recursive relation follows. We call our recursive relation $\text{Pow2}(\mathbf{A}, k)$ and express it as:

$$\text{Pow2}(\mathbf{A}, k) = \begin{cases} \text{Strassen}(\text{Pow2}(\mathbf{A}, k-1), \text{Pow2}(\mathbf{A}, k-1)) & k > 1 \\ \mathbf{A} & k = 1 \end{cases}$$

- (b) The runtime of this algorithm satisfies the following recurrence relation for the variable k :

$$T(k) = T(k-1) + O(n^{2.8074})$$

Assuming that $T(1) = 1$, we get a runtime of $O(kn^{2.8074})$ after unrolling. Since $k = \log_2(m)$, we get a final runtime of $O(\log_2(m)n^{2.8074})$.

- (c) In this case, we can not assume that m is a power of 2. However, we can still make this recursive by cleverly splitting it up for when m is even and odd. If m is even, we can do $\mathbf{A}^m = \mathbf{A}^{0.5m} \mathbf{A}^{0.5m}$ and if m is odd, we can do $\mathbf{A}^m = \mathbf{A} \mathbf{A}^{0.5(m-1)} \mathbf{A}^{0.5(m-1)}$. We call our recursive relation $\text{Pow}(\mathbf{A}, m)$ such that:

$$\text{Pow}(\mathbf{A}, m) = \begin{cases} \text{Strassen}(\text{Pow}(\mathbf{A}, \frac{m}{2}), \text{Pow}(\mathbf{A}, \frac{m}{2})) & m \equiv 0 \pmod{2} \\ \text{Strassen}(\mathbf{A}, \text{Pow}(\mathbf{A}, m-1)) & m \equiv 1 \pmod{2} \end{cases}$$

- (d) The analysis is slightly trickier than before. Assume $T(1) = 1$ and notice that if m is odd, then it takes 2 matrix multiplications to get from $\lfloor \frac{m}{2} \rfloor$ to m . This is our worst case, so we can bound the recurrence by:

$$T(m) \leq T\left(\left\lfloor \frac{m}{2} \right\rfloor\right) + 2O(n^{2.8074})$$

Again using unrolling we see that we achieve a runtime of $O(\log_2(m)n^{2.8074})$. Thus, the asymptotic performance doesn't change.